ChatGPT

Proposed Pipeline for Unsupervised RL-based MITRE Labeling of Procmon Logs

We begin by tokenizing and embedding each raw Procmon log event (process names, file paths, command-line arguments, etc.) into a dense vector. For example, one can use a pre-trained Sentence-BERT model (e.g. `bert-base-nli-mean-tokens` [1] or similar) to encode each event. As in CrowdStrike's command-line detection work, we can then feed these embeddings into lightweight unsupervised anomaly detectors (PCA, Isolation Forest, Autoencoders, etc.) to flag unusual events [2]. In practice this filters the data to the subset of suspicious events, which we will label. Embedding with transformers ensures the text features capture context, and unsupervised outlier models identify which events are likely anomalies [2] [3].

Key Steps: - Tokenize each Procmon record (concatenate relevant fields) and compute a sentence embedding (e.g. via [SentenceTransformers] models) [1] [2].
- Apply unsupervised detectors on the embeddings (e.g. an ensemble of PCA, Isolation Forest, COPOD, Autoencoder) to score anomalies [2]. Retain events above a threshold for further analysis.

Latent Concept Learning and Clustering

Next, we uncover semantic "concepts" in the anomaly events. Following ConceptUML, we apply unsupervised topic modeling to the embeddings to learn latent patterns [4]. For example, Non-negative Matrix Factorization (NMF) can decompose the event-embedding matrix into a set of concept vectors (topics) that represent recurring behaviors [4]. We then model sequences of these concepts using a Hidden Markov Model (HMM) or another sequence clustering method. The HMM groups events (or sequences of events) into clusters by similar latent patterns, automatically selecting the number of clusters via log-likelihood maximization [4].

Key Steps:
- **Concept Extraction:** Run NMF (or LDA) on the BERT embeddings of suspicious events to obtain k latent concepts (topics) representing common actions [4].
- **Sequence Clustering:** Fit an HMM (or Gaussian Mixture) on sequences of concept assignments to cluster related events. Select the optimal number of states by maximizing likelihood [4].
- **Cluster Ranking:** For each cluster, compute a "suspiciousness" score by combining internal consistency and cluster size; identify the most anomalous cluster(s).

By performing NMF+HMM, we automatically group unlabelled events into coherent clusters of behavior (e.g. "file access chain", "registry change + process spawn", etc.) [4].

Incorporating ATT&CK Knowledge (MITRE & CAPEC)

With clusters identified, we map them to ATT&CK techniques using external knowledge. We embed the text descriptions of all relevant MITRE ATT&CK techniques (and optionally CAPEC patterns) into the same semantic space. For each cluster, we compute the cosine similarity between its concept centroid and each technique's embedding [4]. Techniques with high similarity are candidate labels for that cluster.

ConceptUML follows this approach: using cosine-similarity to MITRE/CAPEC corpora to rank and identify the most “suspicious” clusters [4].

Key Steps:
- **Embed Knowledge:** Download open-source ATT&CK technique descriptions (e.g. from MITRE’s website) and embed them (with the same BERT model). Do likewise for CAPEC if used.
- **Similarity Matching:** For each cluster centroid (or example event), find ATT&CK technique vectors with high cosine similarity. These suggest likely labels for events in that cluster.
- **Preliminary Labeling:** Assign to each cluster the top-n matching techniques. This provides a soft multi-label “pseudo-label” based purely on semantics.

This leverages cyber‑security text corpora for unsupervised labeling. In effect we are aligning log concepts with known adversary techniques. (Similar mapping approaches have been used for CVE-to-ATT&CK linkage [5].)

Reinforcement Learning for Label Assignment

We now refine the labeling with reinforcement learning (RL), which can operate without explicit ground-truth labels. We treat each anomalous event (or short event sequence) as a state, and the assignment of one or more ATT&CK labels as the agent’s action. The agent receives a reward based on consistency with the cluster/knowledge cues. For example:

- **State:** the BERT embedding (and possibly contextual info) of the current event or sequence.
- **Action:** select one or more ATT&CK technique IDs. In practice we may formulate this as a multi-step decision (e.g. pick one label at a time) or as a policy outputting a multi-label vector.
- **Reward:** positive if the chosen technique(s) have high semantic similarity to the event (via cosine with knowledge-base embeddings) or match the cluster’s suggested labels; negative otherwise. Additional shaping rewards can encourage diversity or penalize obviously impossible labels.

Over many episodes, the RL agent learns to pick labels that maximize reward. This leverages DRL concepts from recent work: surveys report that deep RL holds promise in intrusion detection tasks [6]. In particular, the survey by Yang et al. recommends combining DRL with knowledge-driven strategies for robust, adaptive detection [6]. Here our “environment” can even be simulated: for example, we can generate pseudo-events from known malicious actions to train the policy.

Key Steps:
- **Define RL Environment:** Each episode presents an event embedding. The agent’s policy (e.g. a DQN or actor-critic network) outputs a technique label or set.
- **Rewards from Knowledge:** Compute reward as, say, the cosine-similarity between the event and chosen technique descriptions (higher means better). We can also reward matches to the cluster’s candidate labels.
- **Training:** Use an RL library (e.g. Stable-Baselines3 or custom PyTorch) to train the policy purely from these intrinsic rewards. No manual labels are needed; the agent learns from “self-supervision” via the ATT&CK knowledge.

Since no ground truth is provided, this RL loop relies on the knowledge-base reward to align labels to events. The unsupervised clusters from before help bootstrap the process by limiting label candidates. In effect, the RL agent implements a labeling policy that is continuously improved by feedback from MITRE semantics.

Enhancing Interpretation with an LLM

To further improve labeling, we can integrate a large language model (LLM) into the loop. An LLM (e.g. GPT-style or Llama) has rich world knowledge and can be prompted for reasoning. For instance, after clustering we might take representative log lines from a cluster and ask the LLM: "Given these system events, which MITRE ATT&CK technique(s) is this most similar to?" The LLM's suggestions can initialize or guide the RL agent's labels. Recent work (e.g. R-Log [7]) shows that combining LLMs with RL yields more accurate log interpretations: the LLM provides candidate answers and the RL fine-tunes the model by rewarding correct inferences.

Key Ideas:
- **LLM Prompting:** Use a pre-trained LLM to generate label hypotheses for clusters or difficult events (e.g. via few-shot prompts). These pseudo-labels enrich the agent's training data.
- **RL Fine-tuning:** Incorporate the LLM into the reward loop: for example, reward labels the LLM also suggests. This mirrors R-Log's "reasoning+RL" scheme where the agent is rewarded for answers aligned with the LLM's reasoning [7].
- **Interpretability:** The LLM can also provide natural-language justifications for a chosen technique, aiding analyst trust.

By combining unsupervised RL with an LLM, we leverage both symbolic ATT&CK knowledge and the reasoning power of a large model. The LLM acts as an "expert oracle" that the RL agent learns from – a promising paradigm in modern log analysis [7].

Outputting the Labeled Dataset

Finally, we produce the labeled dataset while preserving the original logs. Specifically, we attach one or more ATT&CK technique IDs to each anomalous event (or cluster of events) without altering the raw fields. Implementation details on a local machine include:

- **Implementation:** Use Python with open-source libraries. For example, use `pandas` / `numpy` to load Procmon CSVs, `sentence-transformers` (or Hugging Face Transformers) for BERT encoding [1], `scikit-learn` and `hmmlearn` for NMF/HMM, and `stable-baselines3` (or `gym`) for RL.
- **Output Format:** Write the results to a new CSV/JSON where each record has the original log fields plus a "Technique" column (or columns). The new dataset thus contains the same raw events (unchanged) with added labels.
- **Local Execution:** All computation is local. The chosen Sentence-BERT model can be downloaded via `pip`, and the MITRE/CAPEC text is public domain. No proprietary data is needed.

Summary: This unsupervised pipeline embeds Procmon events with BERT, extracts latent concepts (via NMF+HMM), and uses an RL agent to assign ATT&CK labels based on knowledge-base similarity. The approach is motivated by ConceptUML's unsupervised concept learning [4] and by survey recommendations to combine DRL with domain knowledge [6]. It yields a fully labeled anomaly dataset without any manual annotation, mapping raw log events directly to MITRE techniques.

Sources: We base this design on recent literature: ConceptUML's use of BERT, NMF, HMM and MITRE/CAPEC similarity [4] [1]; BERT-driven anomaly detection without labels [2] [3]; DRL surveys on intrusion detection [6]; and LLM+RL log analysis methods [7]. These inform each pipeline component.

---

1 4 GitHub - khanhluongds/ConceptUML_Multiphase_Unsupervised_Threat_Detection
https://github.com/khanhluongds/ConceptUML_Multiphase_Unsupervised_Threat_Detection

2 3 BERT Embeddings: A Modern Machine-learning Approach for Detecting Malware from Command Lines (Part 1 of 2)
https://www.crowdstrike.com/en-us/blog/bert-embeddings-new-approach-for-command-line-anomaly-detection/

5 CVE2ATT&CK: BERT-Based Mapping of CVEs to MITRE ATT&CK Techniques | MDPI
https://www.mdpi.com/1999-4893/15/9/314

6 [2410.07612] A Survey for Deep Reinforcement Learning Based Network Intrusion Detection
https://arxiv.org/abs/2410.07612

7 R-Log: Incentivizing Log Analysis Capability in LLMs via Reasoning-based Reinforcement Learning
https://arxiv.org/html/2509.25987v1